

实验报告：中间代码生成模块实现

姓名：吴季鸿 学号：241220142

1. 已实现功能

本次实验在已有词法分析、语法分析和语义分析程序的基础上，新增实现了 **C-- 到三地址中间代码 (IR) 的生成模块**。程序整体流程为：先完成词法、语法和语义检查；若输入程序无错误，且属于当前 IR 支持的语言子集，则输出对应的中间代码文件。

本次实现的 IR 生成功能主要包括：

- **函数相关翻译**：支持函数定义、函数调用、参数传递和返回值处理，生成 **FUNCTION**、**PARAM**、**ARG**、**CALL**、**RETURN** 等指令。
- **局部变量与数组声明翻译**：支持整型变量和数组变量声明；数组使用 **DEC** 指令分配空间，并递归计算数组宽度。
- **表达式翻译**：支持整型常量、普通变量、赋值表达式、四则运算、括号表达式、单目负号等。
- **控制流翻译**：支持 **if**、**if-else** 和 **while**，生成 **LABEL**、**GOTO** 和条件跳转指令。
- **数组访问翻译**：支持一维与多维数组元素访问，通过“基地址 + 偏移量”的方式完成寻址。
- **内建函数翻译**：支持 **read()** 与 **write(x)**，分别翻译为 **READ** 与 **WRITE** 指令。

此外，为保证输出结果更紧凑、可测试，在生成 IR 时还做了若干基础优化，例如：

- 条件判断中直接使用简单操作数，减少冗余临时变量；
- 表达式语句不保留无用返回值；
- 对 **if-else** 中以 **return** 结束的分支避免生成多余跳转；
- 对简单实参与初始化表达式优先直接使用操作数表示。

2. 主要实现思路

本次实验新增内容主要集中在 **ir.c** 中，采用了 **递归遍历语法树并同步输出 IR** 的实现方式。其核心思路如下。

2.1 简化类型系统与局部符号表

为配合 IR 生成，在 **ir.c** 中单独维护了一套简化类型系统，仅保留当前实验真正需要的三类类型：整型、数组和函数。与此同时，还维护了全局函数表和当前函数的局部变量表，用于完成：

- 标识符到类型信息的映射；
- 数组宽度计算；
- 区分形参与普通局部变量；
- 函数调用时查询参数和返回值信息。

这样做的好处是 IR 模块可以独立工作，不必直接依赖语义分析阶段的完整类型系统，结构更清晰，调试也更方便。

2.2 表达式翻译

表达式翻译函数 **translate_exp** 是整个 IR 生成的核心。设计上区分了两种情况：

- 若表达式需要值，则把结果放入目标位置 **place**；

- 若表达式只作为语句出现，则允许 `place = NULL`，只保留副作用。

为了减少机械式中间变量，实现中引入了 `translate_operand`：

- 对变量和整型常量，直接返回操作数形式；
- 对复杂表达式，先生成临时变量保存结果，再返回该临时变量。

这一设计使条件判断、函数实参与数组下标等场景中的代码明显更紧凑。

2.3 控制流翻译

控制流部分采用经典三地址代码方法实现。布尔表达式由 `translate_cond` 翻译成真假出口跳转；`if-else` 和 `while` 则通过 `LABEL` 与 `GOTO` 组织控制流。

在此基础上，实现中还增加了 `stmt_ends_control` 来判断某条语句是否必然以 `return` 结束。若 `if-else` 的某个分支已经必然结束控制流，就不再补充跳往公共出口的 `GOTO`，从而去掉了大量明显冗余的跳转代码。

2.4 数组翻译

数组翻译是本次实现中较有代表性的部分。程序将多维数组统一看作递归嵌套的数组类型，并利用 `type_width` 递归计算每一层元素宽度。数组引用翻译时：

1. 先确定基地址；
2. 将当前维度下标乘以该层元素宽度得到偏移量；
3. 将偏移量与已有地址累加。

因此，一维数组和多维数组在实现上被统一成“递归求地址”的过程，逻辑简洁且便于扩展。

2.5 支持范围控制

本次提交的 IR 生成器仅面向当前要求范围内的语言子集。对于超出支持范围的输入程序，不生成中间代码文件。这样可以保证所有输出结果都来自已经完成并验证过的翻译功能，避免对未实现部分给出错误的 IR。

3. 编译与运行方式

本实验代码可在 Linux 环境下使用 `make` 编译。推荐编译方式如下：

```
make clean
make
```

编译成功后会生成可执行文件 `parser`。运行方式为：

```
./parser input.cmm output.ir
```

其中：

- `input.cmm` 为待翻译的 C- 源文件；
- `output.ir` 为输出的中间代码文件。

若源程序存在词法、语法或语义错误，或者不属于当前 IR 支持的语言子集，则不会生成有效的中间代码输出文件。

4. 本次实现的亮点

本次实验最主要的新工作是 **IR 中间代码生成模块的完整搭建**。相较于简单地将语法树机械翻译成字符串，本实现更关注输出代码的结构性和可读性，主要亮点包括：

- 采用独立的 IR 类型和符号管理，使模块边界清晰；
- 通过 `translate_operand` 统一简单操作数和复杂表达式的处理，显著减少冗余临时变量；
- 通过 `stmt_ends_control` 对控制流输出做基础优化，避免 `RETURN` 后的无意义跳转；
- 对数组采用统一的递归地址计算方法，使一维和多维数组访问共享同一套翻译框架。

总体来看，在本次实验中已经完成了中间代码生成的主体实现，并在表达式、控制流和数组翻译三个部分做了较系统的设计与优化。